COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010	**SECTION:**	CPE21S2
DATE PERFORMED:	7/17/26	**DATE SUBMITTED:**	7/22/26
NAME:	SUBALISID, JAPPETH ROYCE B.	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. HOA 2.1 Arrays, Pointers and Dynamic Memory Allocation

1. PROCEDURE OUTPUT

ILO A: Implement static and dynamic memory allocation & ILO B: Create dynamically allocated objects using pointers and arrays

Screenshot

```cpp
#include <iostream>
#include <string.h>

class Student{
    private:
        std::string studentName;
        int studentAge;
    public:
        //constructor
        Student(std::string newName = "Jane Doe", int newAge = 18) {
            studentName = std::move(newName);
            studentAge = newAge;
            std::cout << "Constructor called!\n";
        };

        //deconstructor
        ~Student() {
            std::cout << "Deconstructor called!\n";
        }

        //copy constructor
        Student(const Student &copyStudent) {
            std::cout << "Copy constructor called!\n";
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }

        //display attributes
        void printDetails() {
            std::cout << this->studentName << " " << this->studentAge << '\n';
        }

};

int main() {

    Student student1("Ezio", 60);
    Student student2(student1);
    Student student3;
    student3 = student2;

    return 0;
}
```

```
Constructor called!
Copy constructor called!
Constructor called!
Deconstructor called!
Deconstructor called!
Deconstructor called!
```

```
student1.printDetails();
student2.printDetails();
student3.printDetails();
```

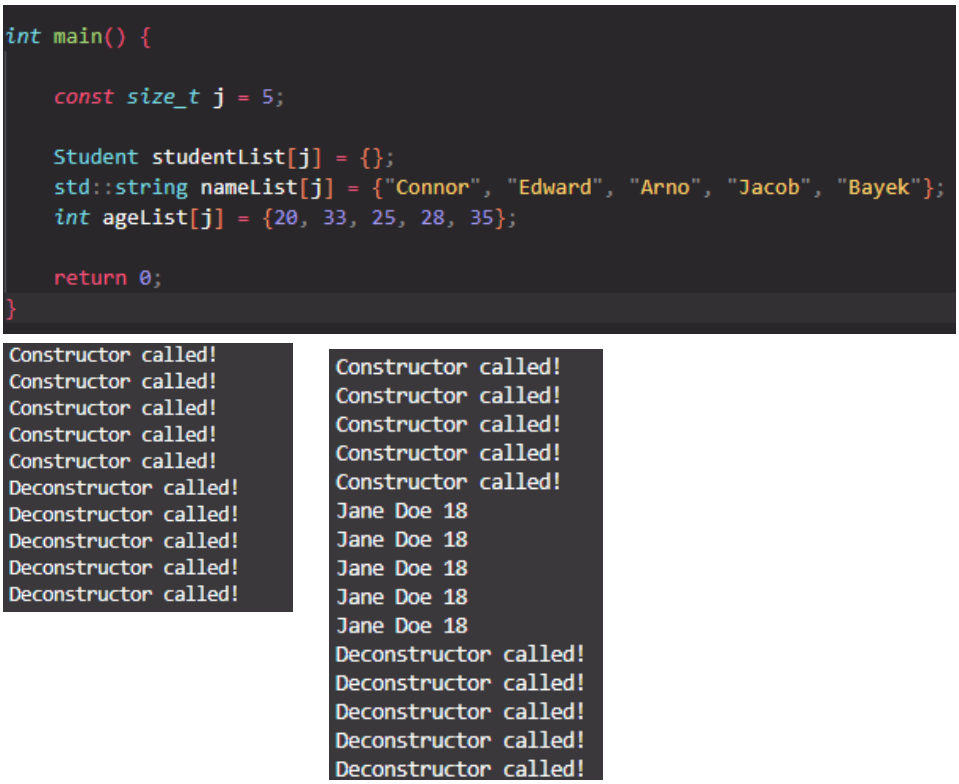
```
Constructor called!
Copy constructor called!
Constructor called!
Ezio 60
Ezio 60
Ezio 60
Deconstructor called!
Deconstructor called!
Deconstructor called!
```

```
//copy assignment operator
Student& operator=(const Student &caoStudent) {
    std::cout << "Copy assignment operator called!\n";
    if(this != &caoStudent) {
        studentName = caoStudent.studentName;
        studentAge = caoStudent.studentAge;
    }
    return *this;
}
```

```
Constructor called!
Copy constructor called!
Constructor called!
Copy assignment operator called!
Destructor called!
Destructor called!
Destructor called!
```

Observation	The terminal shows 3 constructor calls. The first one creates a student based on the given arguments("Ezio" and 60). The second one runs the copy constructor method and creates a student copying the details of the first student. The third one creates a default parameter student ("Jane Doe" and 18). The student3 = student2 assignment doesn't print anything since there is no 'cout' associated with it. Then lastly the deconstructor runs 3 times since there is also 3 instances of the student object. Adding the printDetails function call to main prints 3 similar objects, meaning that the copy assignment operator ran successfully. Adding a copy assignment operator shows how the program does the copying with an equal sign instead of using it as an argument. The method just modifies how the operator '=' acts when it is used on the object.

Table 2-1. Initial Driver Program

Screenshot	
Observation	After running the program, we can see that the constructor and deconstructor both ran 5 times, but after printing the details (using loop B), we can see that all objects have default parameters. This is because the program only created the objects on the studentList[j] array which have no arguments.

```
int main() {

    const size_t j = 5;

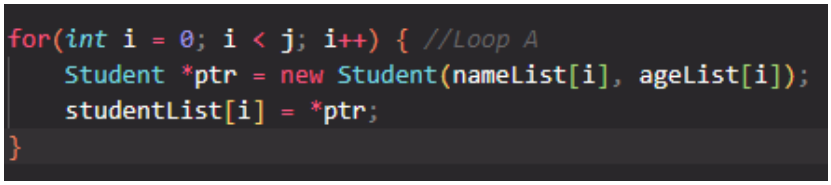
    Student studentList[j] = {};
    std::string nameList[j] = {"Connor", "Edward", "Arno", "Jacob", "Bayek"};
    int ageList[j] = {20, 33, 25, 28, 35};

    return 0;
}
```

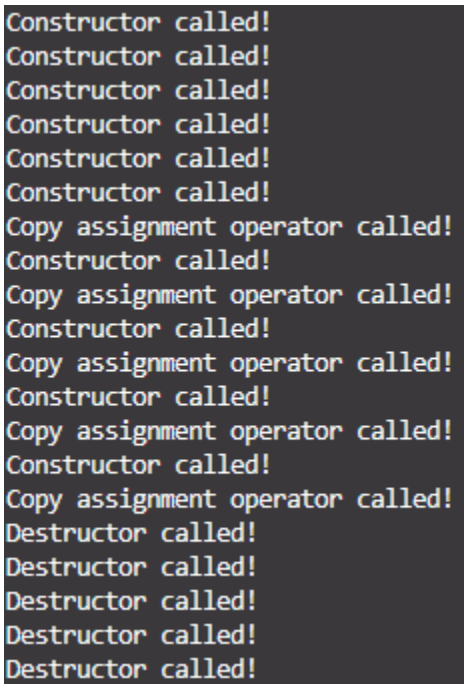
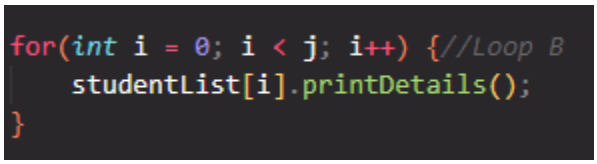
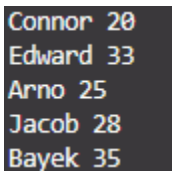
```
Constructor called!
Constructor called!
Constructor called!
Constructor called!
Constructor called!
Deconstructor called!
Deconstructor called!
Deconstructor called!
Deconstructor called!
Deconstructor called!
```

```
Constructor called!
Constructor called!
Constructor called!
Constructor called!
Constructor called!
Jane Doe 18
Jane Doe 18
Jane Doe 18
Jane Doe 18
Jane Doe 18
Deconstructor called!
Deconstructor called!
Deconstructor called!
Deconstructor called!
Deconstructor called!
```

Table 2-2. Modified Driver Program with Student Lists

Loop A	

```
for(int i = 0; i < j; i++) { //Loop A
    Student *ptr = new Student(nameList[i], ageList[i]);
    studentList[i] = *ptr;
}
```

	
Observation	The constructor method seems to have run 10 times after running Loop A, I assume this is because of new assignments. The loop first creates a student object on the heap (with arguments of Connor and 20), each calling the constructor method, then it accesses the studentList[i], the first object on the array, and assigns the value from the pointed object in the heap(with arguments Connor and 20), calling the copy assignment operator after each one.
Loop B	 
Observation	After running Loop B, it prints all the details of the objects on the list. Now it has changed from default parameters to the listed argument values.

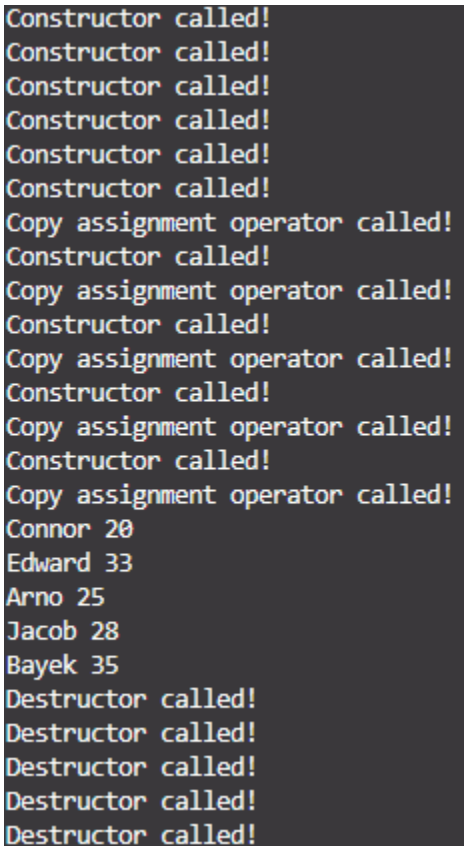
Output	
Observation	The program runs and constructs 5 default parameter objects. Then creates the new objects on the heap, calling constructor again, then copies the new object's values to the old default parameter objects. After that, loop b prints all the object's details, then the destructor runs.

Table 2-3. Final Driver Program

Modifications	
Observations	

Table 2-4. Modifications/Corrections Necessary

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve programming problems using dynamic memory allocation, arrays and pointers

Jenna's Grocery List		
Apple	PHP 10	x7
Banana	PHP 10	x8
Broccoli	PHP 60	x12
Lettuce	PHP 50	x10

Jenna wants to buy the following fruits and vegetables for her daily consumption. However, she needs to distinguish between fruit and vegetable, as well as calculate the sum of prices that she has to pay in total.

Problem 1: Create a class for the fruit and the vegetable classes. Each class must have a constructor, deconstructor, copy constructor and copy assignment operator. They must also have all relevant attributes (such as name, price and quantity) and functions (such as calculate sum) as presented in the problem description above.

```cpp
class Item{ //parent class
    protected: // default members for children
        std::string itemName;
        float itemPrice;
        int itemQuantity;
    public:
        //constructor
        Item(std::string newName = "", float newPrice = 0.0, int newQuantity = 0) {
            itemName = std::move(newName);
            itemPrice = newPrice;
            itemQuantity = newQuantity;
            std::cout << "Item constructor called!\n";
        }

        //destructor
        virtual ~Item() { //virtual to clean child class along with parent
            std::cout << "Deleting item '" << itemName << "'\n";
        }

        Item(const Item& other) { //copy constructor for item class
            itemName = other.itemName;
            itemPrice = other.itemPrice;
            itemQuantity = other.itemQuantity;
            std::cout << "Item Copy Constructor called\n";
        }

        Item& operator=(const Item& other) { //copy assignment operator for item class
            std::cout << "Item Copy Assignment Operator called\n";
            if (this != &other) {
                itemName = other.itemName;
                itemPrice = other.itemPrice;
                itemQuantity = other.itemQuantity;
            }
            return *this;
        }

        float sumPrice() { //basic multiplication for price of each item type
            return this->itemPrice * this->itemQuantity;
        }

        void printDetails() { //print details for each item
            std::cout << std::left << std::setw(15) //formatting for nice output
            << this->itemName
            << "\tPhp: " << this->itemPrice
            << "\tAmount: x" << this->itemQuantity
            << "\tTotal Price: Php " << this->sumPrice() << '\n';
        }
};
```

```cpp
class Fruit : public Item { //child class for fruit
public:
    Fruit(std::string newName = "", float newPrice = 0.0, int newQuantity = 0) {//constructor for fruit class
        itemName = std::move(newName);
        itemPrice = newPrice;
        itemQuantity = newQuantity;
        std::cout << "Fruit constructor called!\n";
    }

    ~Fruit() override { //override to clean child class along with parent
        std::cout << "Fruit destructor called!\n";
    }

    Fruit(const Fruit& other) : Item(other) { //copy constructor for fruit class
        std::cout << "Fruit Copy Constructor called\n";
    }

    Fruit& operator=(const Fruit& other) { //copy assignment operator for fruit class
        std::cout << "Fruit Copy Assignment Operator called\n";
        if (this != &other) {
            Item::operator=(other);
        }
        return *this;
    }

    float fruitTotal() { //function to calculate total price of fruits
        return this->itemPrice * this->itemQuantity;
    }
};
```

```cpp
class Vegetable : public Item { //child class for vegetable
public:
    Vegetable(std::string newName = "", float newPrice = 0.0, int newQuantity = 0) {//constructor for vegetable class
        itemName = std::move(newName);
        itemPrice = newPrice;
        itemQuantity = newQuantity;
        std::cout << "Vegetable constructor called!\n";
    }

    ~Vegetable() override { //override to clean child class along with parent
        std::cout << "Vegetable destructor called!\n";
    }

    Vegetable(const Vegetable& other) : Item(other) { //copy constructor for vegetable class
        std::cout << "Vegetable Copy Constructor called\n";
    }

    Vegetable& operator=(const Vegetable& other) { //copy assignment operator for vegetable class
        std::cout << "Vegetable Copy Assignment Operator called\n";
        if (this != &other) {
            Item::operator=(other);
        }
        return *this;
    }

    float vegetableTotal() { //function to calculate total price of vegetables
        return this->itemPrice * this->itemQuantity;
    }

};
```

Problem 2: Create an array GroceryList in the driver code that will contain all items in Jenna's Grocery List. You must then access each saved instance and display all details about the items.

```
int main() {
    Item* GroceryList[4] = {}; //array of pointers to Item objects

    GroceryList[0] = new Fruit("Apple", 10, 7); //creating objects in the heap and assigning them to the array of pointers
    GroceryList[1] = new Fruit("Banana", 10, 8);
    GroceryList[2] = new Vegetable("Broccoli", 60, 12);
    GroceryList[3] = new Vegetable("Lettuce", 50, 10);

    //Printing the details of each item in the grocery list
    std::cout << std::endl;
    for (int i = 0; i < 4; i++) {
        GroceryList[i]->printDetails();
    }
```

Problem 3: Create a function TotalSum that will calculate the sum of all objects listed in Jenna's Grocery List.

```
float TotalSum(Item* GroceryList[]) { //function TotalSum that will calculate the sum of all objects
    float totalPrice = 0.0;
    for(int i = 0; i < 4; i++) {
        totalPrice += GroceryList[i]->sumPrice();
    }
    return totalPrice;
}
```

```
//Total price
std::cout << "\nTotal Price: Php " << TotalSum(GroceryList) << '\n';
```

Problem 4: Delete the Lettuce from Jenna's GroceryList list and de-allocate the memory assigned.

```
//Delete the Lettuce from Jenna's GroceryList list and de-allocate the memory assigned.
if (GroceryList[3] != nullptr) {
    delete GroceryList[3];
    GroceryList[3] = nullptr;
}
```

Deleting the other objects:

```
std::cout << std:: endl;
for (int i = 0; i < 4; i++) {//deleting the objects in the heap and de-allocating the memory assigned
    if (GroceryList[i] != nullptr) {
        delete GroceryList[i];
        GroceryList[i] = nullptr;
    }
}
```

```
Item constructor called!
Fruit constructor called!
Item constructor called!
Fruit constructor called!
Item constructor called!
Vegetable constructor called!
Item constructor called!
Vegetable constructor called!

Apple           Php: 10 Amount: x7      Total Price: Php 70
Banana          Php: 10 Amount: x8      Total Price: Php 80
Broccoli        Php: 60 Amount: x12     Total Price: Php 720
Lettuce         Php: 50 Amount: x10     Total Price: Php 500

Total Price: Php 1370
Vegetable destructor called!
Deleting item 'Lettuce'

Fruit destructor called!
Deleting item 'Apple'
Fruit destructor called!
Deleting item 'Banana'
Vegetable destructor called!
Deleting item 'Broccoli'
```

- Summary of lessons learned
  - *I learned how to assign arguments to objects directly and using a list. I also learned how to use the pointers to create objects directly in the heap. I also learned how the constructors work in objects.*
- Analysis of the procedure
  - *The procedure activities were easy to follow but I had to learn how the functions work on my own since I wanted to understand them deeper so that I can use them on my own in future projects without relying on provided code*
- Analysis of the supplementary activity
  - The supplementary activity was a bit difficult since I had to create a class for two classes(Inheritance in OOP) so that I could make an array that contains two object types(Fruit and vegetable). It was hard to understand how it works but I just learned the essential parts that I needed in the requirements.
- Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?
  - I think I need more practice in using the classes in c++ since I don't know the ins and outs of OOP in it. I also need to inform myself of the purpose of doing these things so that I understand the need to master the skill of doing these kinds of activities.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

COMPUTER ENGINEERING

QUEZON CITY

Gemini

Purpose of Use:

Learning to code OOP in C++, syntax correction, formatting

Extent of Use:

Used to explore how to do Inheritance of classes in c++, checked how to format output in iomanip

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.